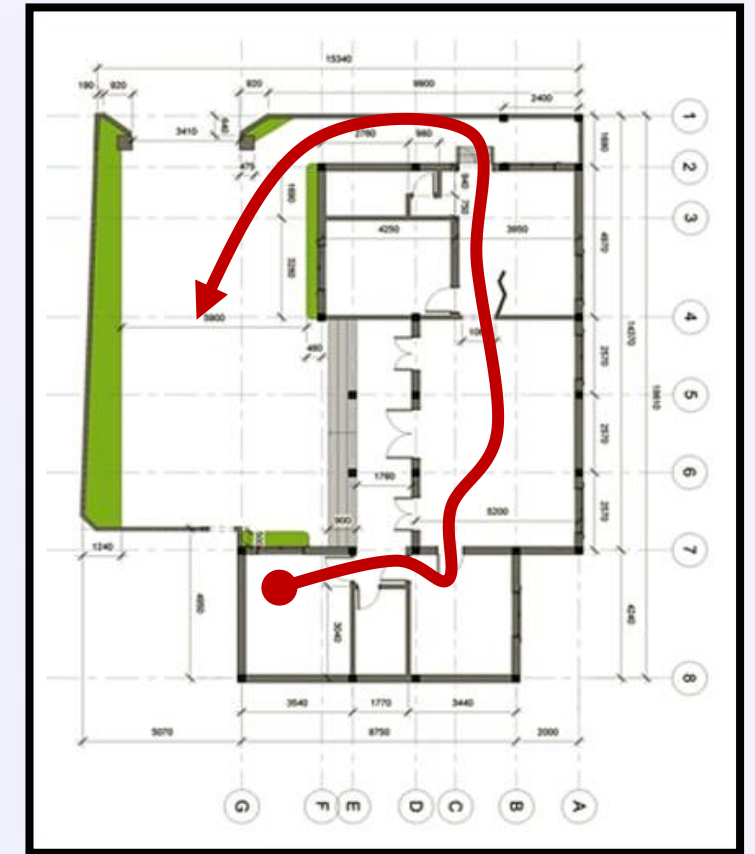


Path Planning on a Graph

Dr. Troi Williams

Slide credits: Pratap Tokekar, Chahat Deep Singh

Find a path from point A to B



assuming we know the map

This lecture (and next)

(Five) Assumptions

- Known map
- No moving obstacles
- Point robot
- Perfect sensing (we know where we are at all times)
- Discrete time, discrete state, discrete actions

Properties of a good planner

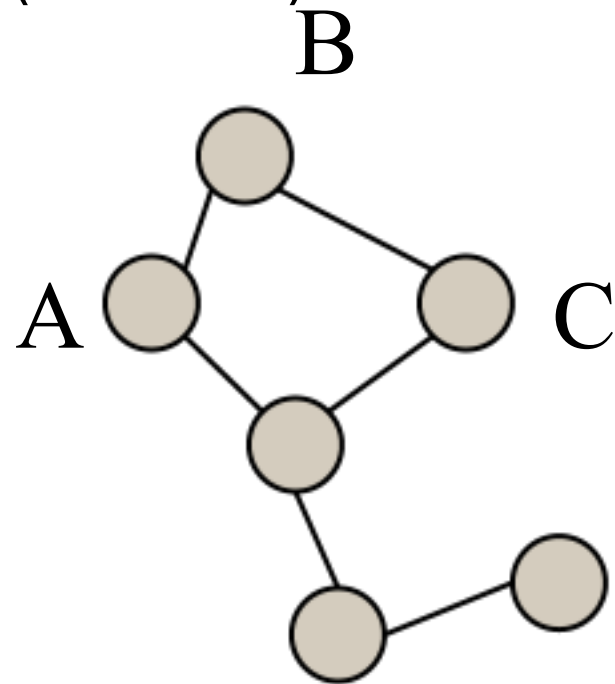
1. *Completeness*: Planner is complete if it **always** finds **a path** from start to goal in finite time, if one exists. Else, it declares failure in finite time.
2. *Optimality*: Planner is optimal if it always finds a **minimum cost path** from start to goal in finite time, if one exists.
3. *Efficiency*: An algorithm is efficient if it finds the path in the **least possible computational time** (for all inputs).

Our plan

- Today: *complete* but not *optimal* planners
 - Breadth First Search (BFS)
 - Depth First Search (DFS)
- Next lecture: *optimal* and *efficient* planners
 - Dijkstra
 - A*
- Later: *complete* but better running time than BFS/DFS for more complex settings
 - RRT

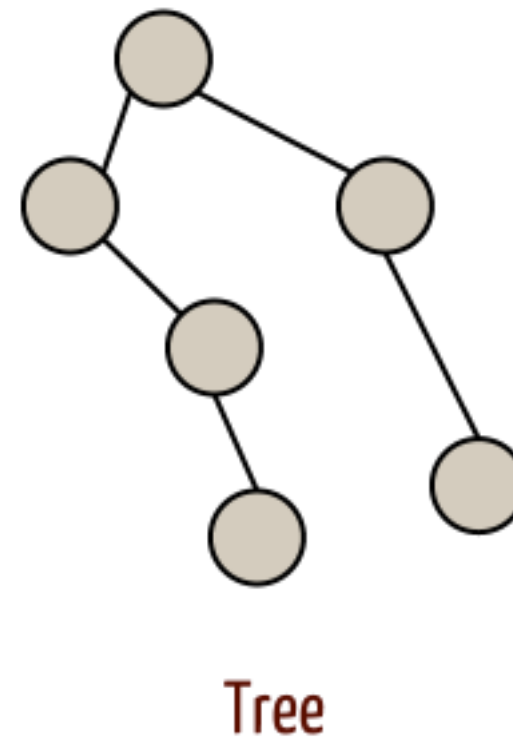
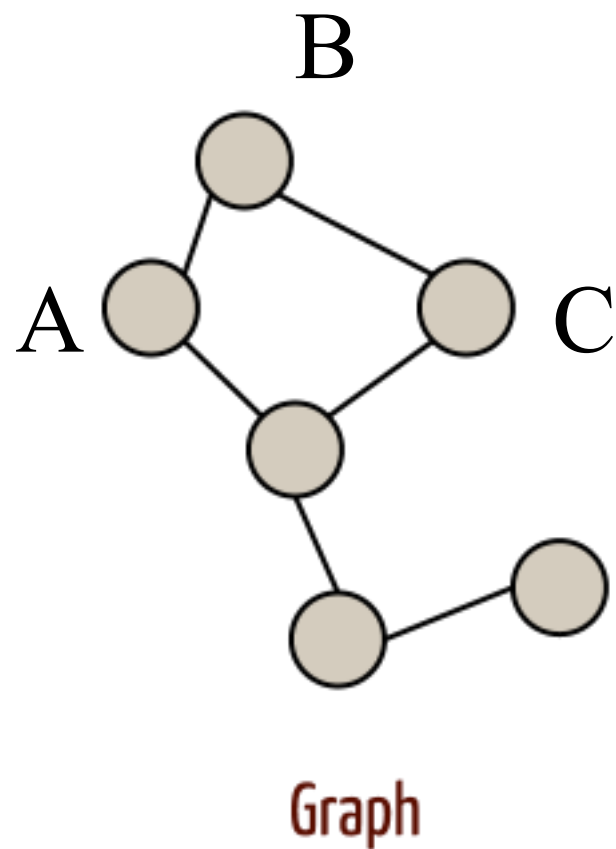
Graphs

- Mathematical structures to model **pairwise relations between objects**
- Graph consists of
 - **Vertices** (aka nodes)
 - **Edges** (aka links)



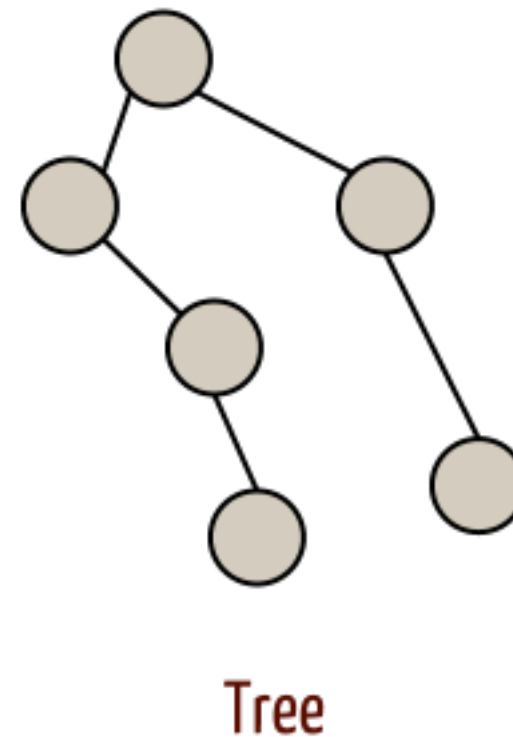
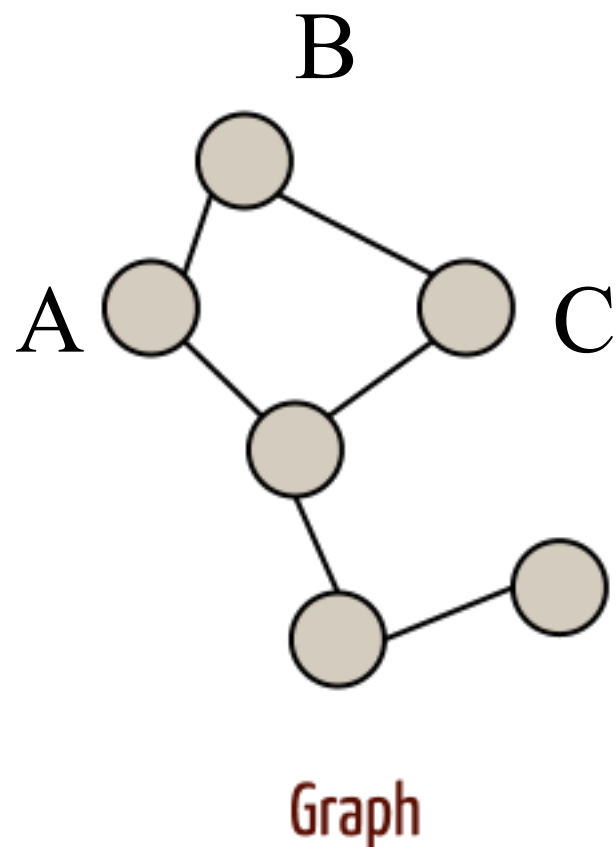
Graphs versus Trees

- What are trees in the context of graphs?



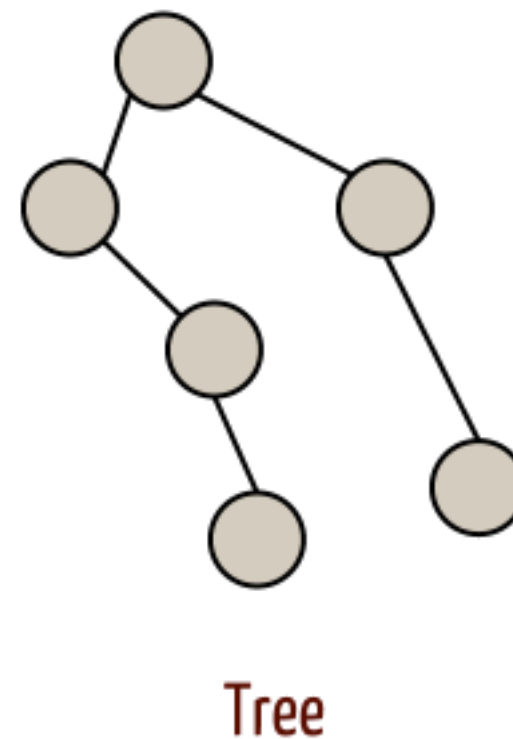
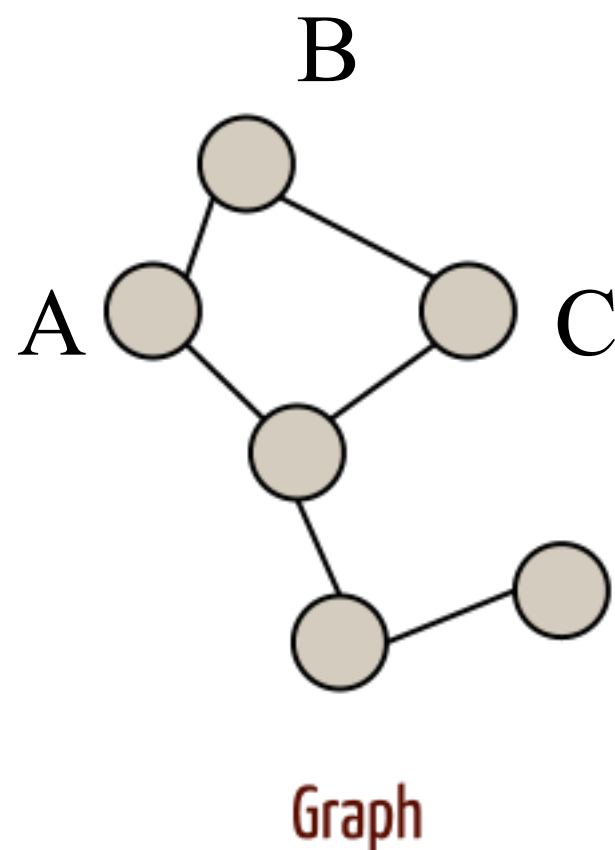
Graphs versus Trees

- Trees are graphs without any *cycle*
- All trees are graphs but not all graphs are trees



Graphs versus Trees

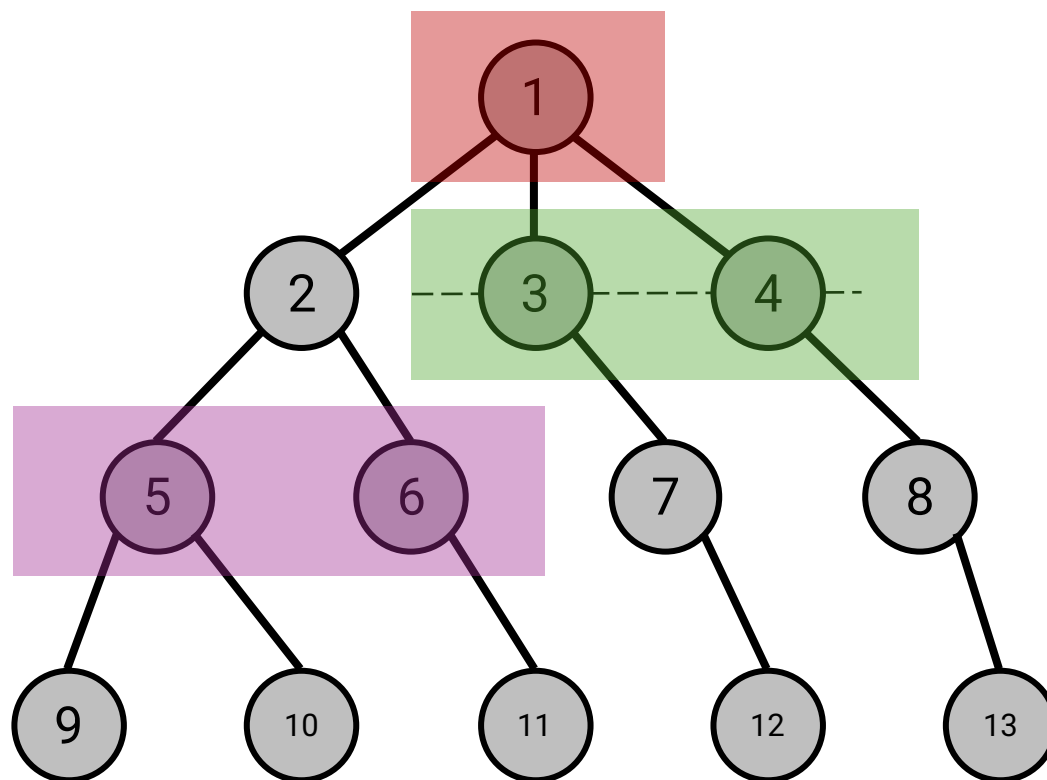
- Trees are graphs without any cycle
- All trees are graphs, but not all graphs are trees



- *We'll make the connection between graphs and trees more explicit later.*

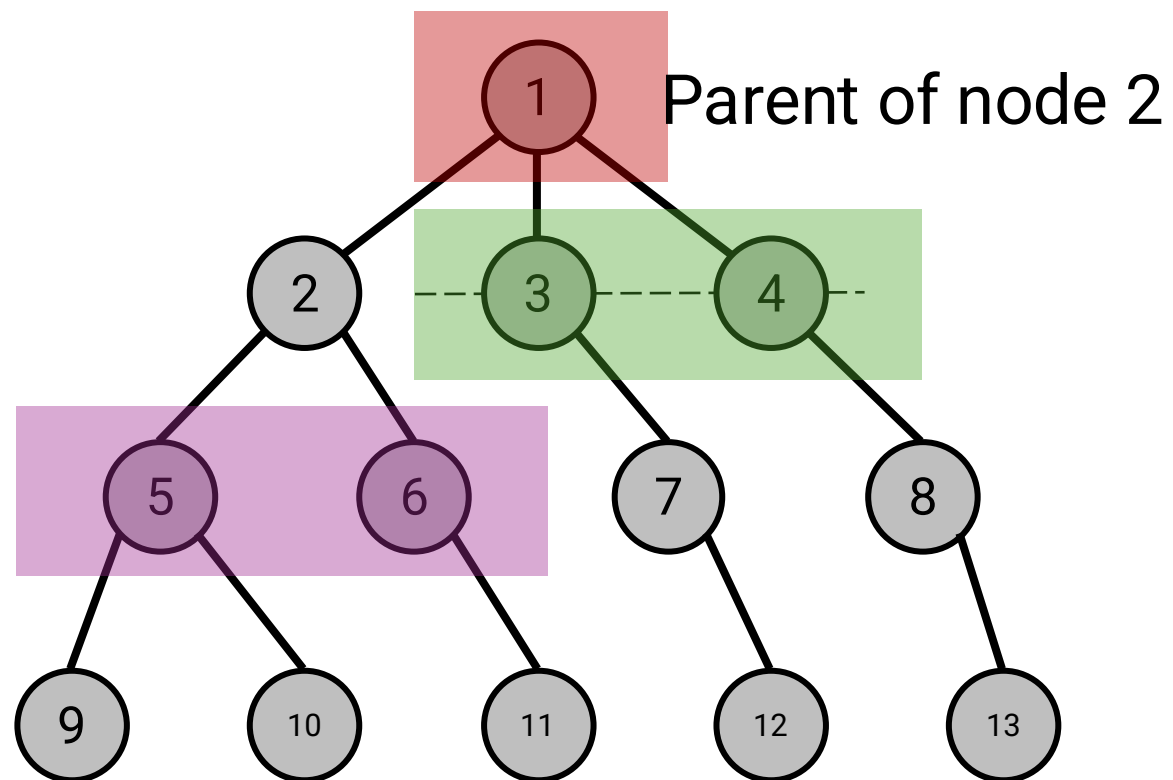
Trees

Consider node 2



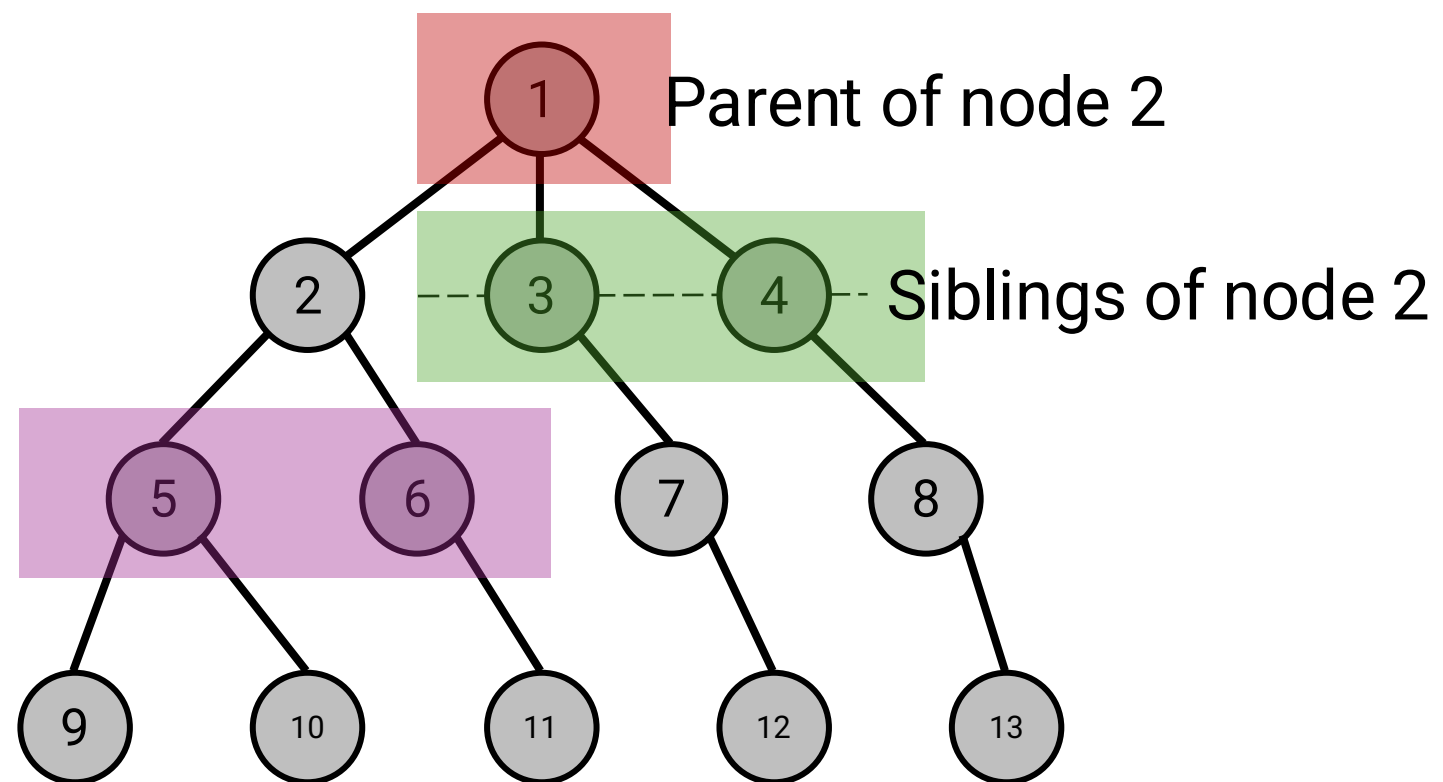
Trees

Consider node 2



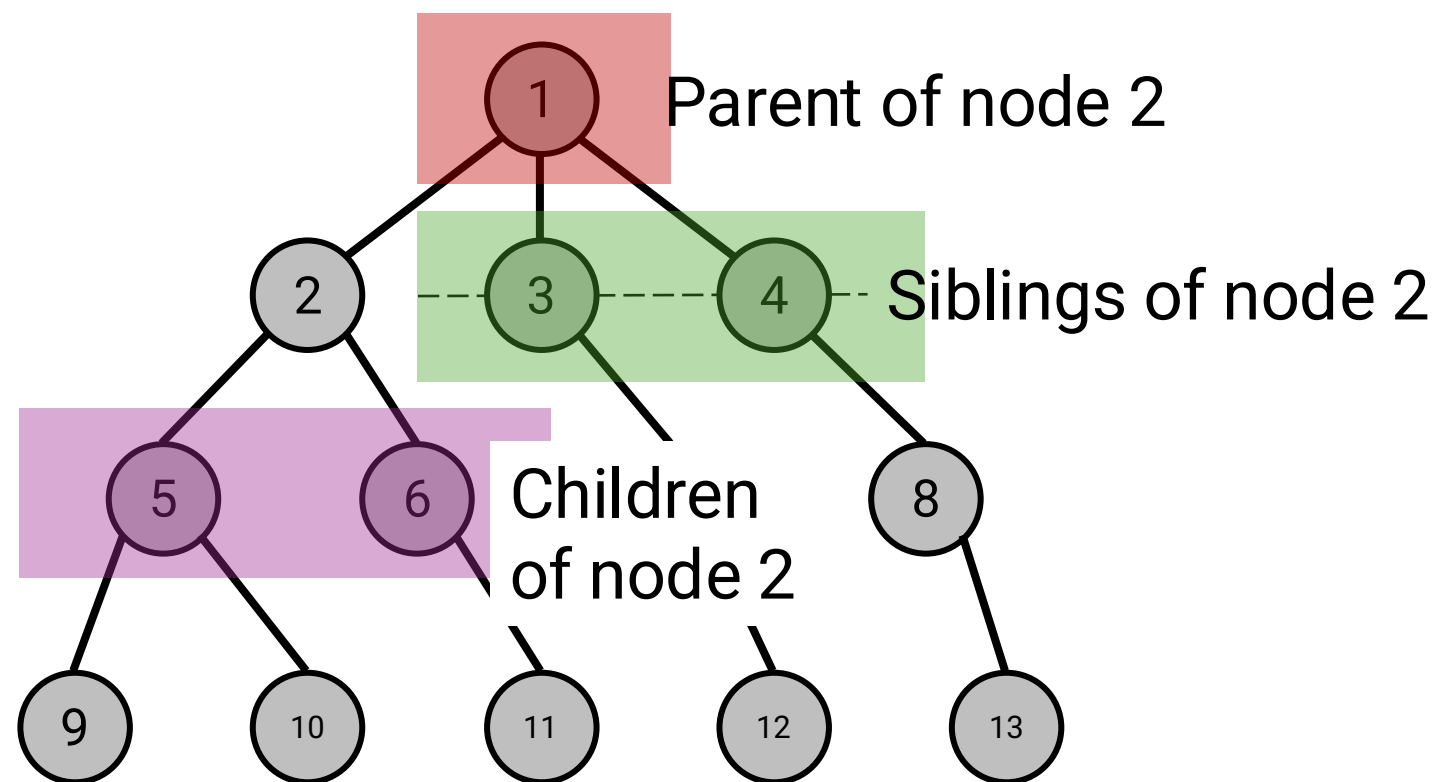
Trees

Consider node 2



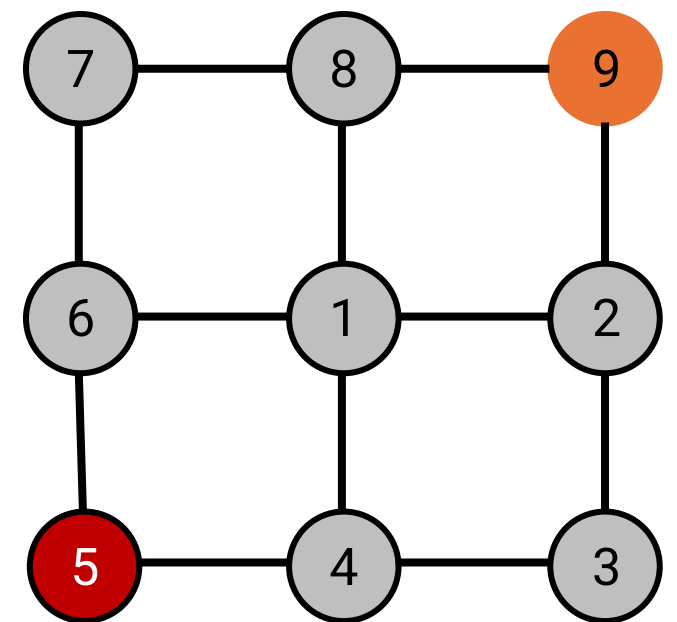
Trees

Consider node 2



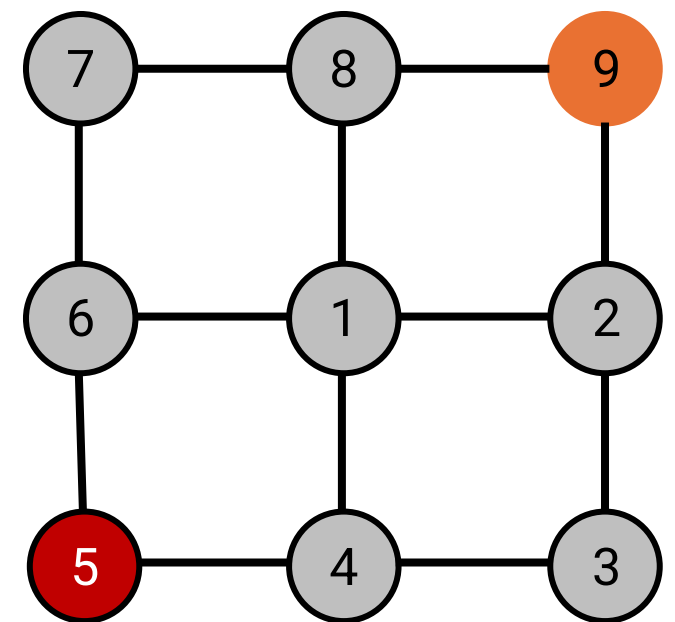
How does this work on a graph?

- Find a path from 5 to 9
- Use DFS
- Need to decide which order to add children nodes in
- Need to avoid cycles
- The subsequent slides will show the DFS output as a tree with (S) denoting the stack and (V) denoting the visited nodes.



How does this work on a graph?

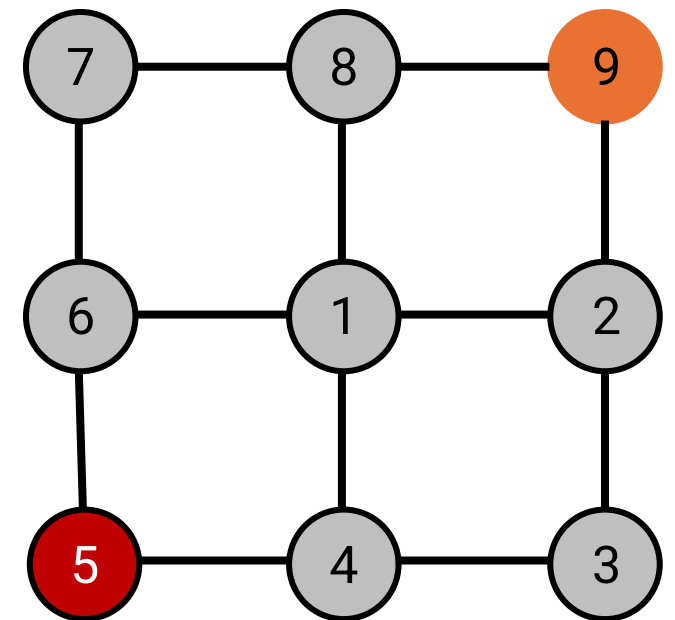
- Data structures used:
 - **Stack** to store the list of vertices to explore. A stack operates as a Last-In, First-Out data structure. (Here's a [YouTube video on stacks](#)).
 - **Set** to maintain the vertices we've visited (i.e., the vertices we've removed from the stack to explore).
 - **Tree** to store the final path from the start vertex to the goal vertex. *At the end, you'll see why we use a tree to find the path.
- Exploration order: Right, down, left, up.



How does this work on a graph?

Algorithm steps:

1. Add the start vertex to the stack.
2. Loop while your stack *S* is not empty:
 - a. Remove vertex *X* from the **top of the stack *S*** (the last vertex you added).
 - b. If you've visited *X* before (i.e., *X* is in your visited set *V*), go to step 2a.
 - c. **Add *X* to your visited set *V*.**
 - d. If *X* is your goal, exit the loop!
 - e. Find all the **unvisited neighbors of *X*** and **add them to the stack** (remember the explore order).
 - f. **Add neighbor *C* under *X*** in the tree if *C* is not on the stack *S* or in the visited set *V*.

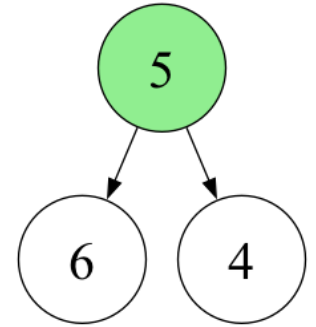


How does this work on a graph?

- Start with node 5 on the stack
(initialization).

- S: 5
- V:

How does this work on a graph?

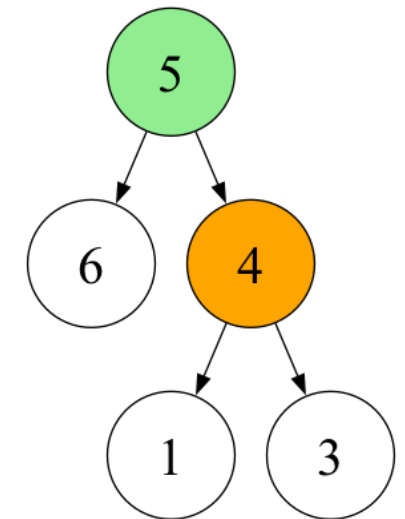


Step 1: Exploring Node 5

- Remove 5 from the stack S and check if it is the goal.
- It's not, so add it to the visited set V .
- Add unvisited neighbors of 5 to stack: 6, 4
- Add 6, 4 as children of 5 in tree.
- S : 6, 4
- V : 5

How does this work on a graph?

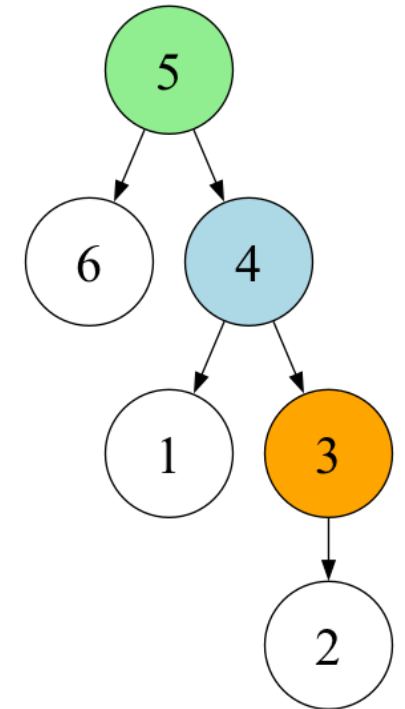
- Remove 4 from the stack S and check if it is the goal.
- It's not, so add it to the visited set V.
- Add unvisited neighbors of 4 to stack: 1, 3
- Add 1, 3 as children of 4 in tree.
- S: 6, 1, 3
- V: 5, 4



Step 2: Exploring Node 4

How does this work on a graph?

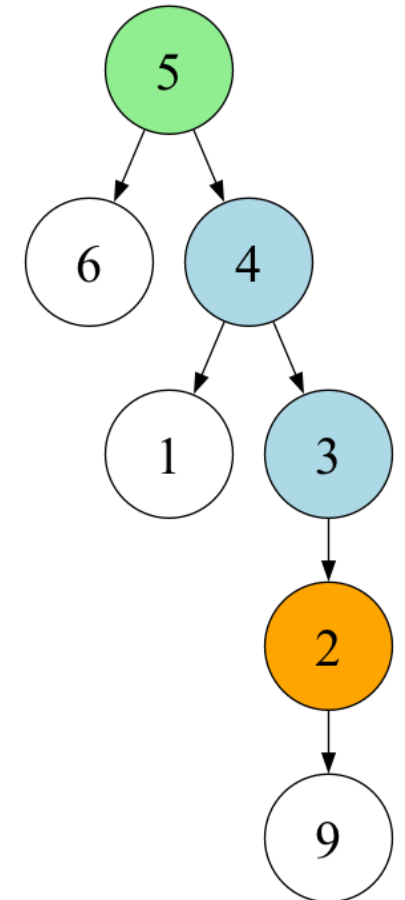
- Remove 3 from the stack S and check if it is the goal.
- It's not, so add it to the visited set V.
- Add unvisited neighbors of 3 to stack: 2
- Add 2 as child of 3 in tree.
- S: 6, 1, **2**
- V: 5, 4, **3**



Step 3: Exploring Node 3

How does this work on a graph?

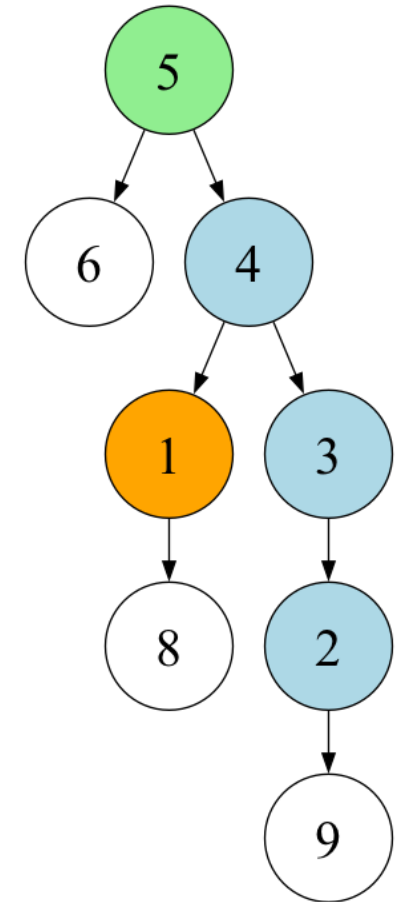
- Remove 3 from the stack S and check if it is the goal.
- It's not, so add it to the visited set V.
- Add unvisited neighbors of 2 to stack: 9, 1
- Add 9 as child of 2 in tree.
- S: 6, 1, 3, **9**, **1**
- V: 5, 4, 3, **2**



Step 4: Exploring Node 2

How does this work on a graph?

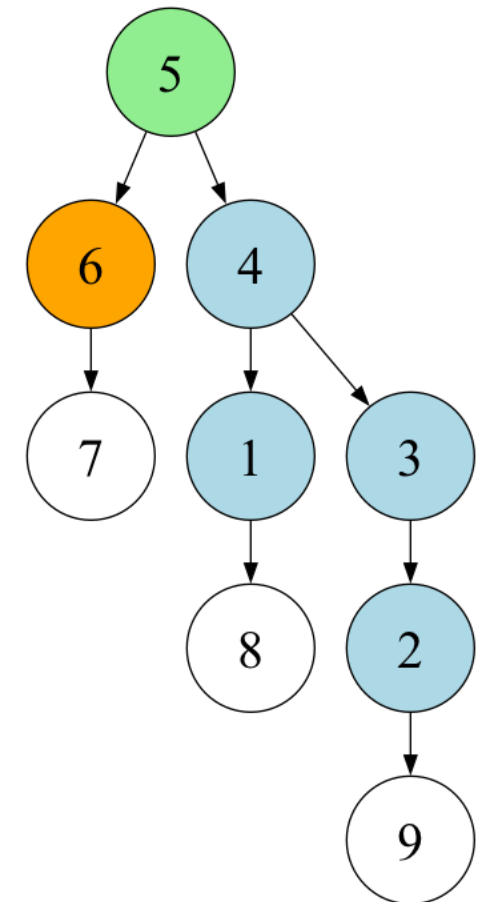
- Remove 1 from the stack S and check if it is the goal.
- It's not, so add it to the visited set V.
- Add unvisited neighbors of 1 to stack: 8, 6
- Add 8 as child of 1 in tree.
- S: 6, 1, 3, 9, **8, 6**
- V: 5, 4, 3, 2, **1**



Step 5: Exploring Node 1

How does this work on a graph?

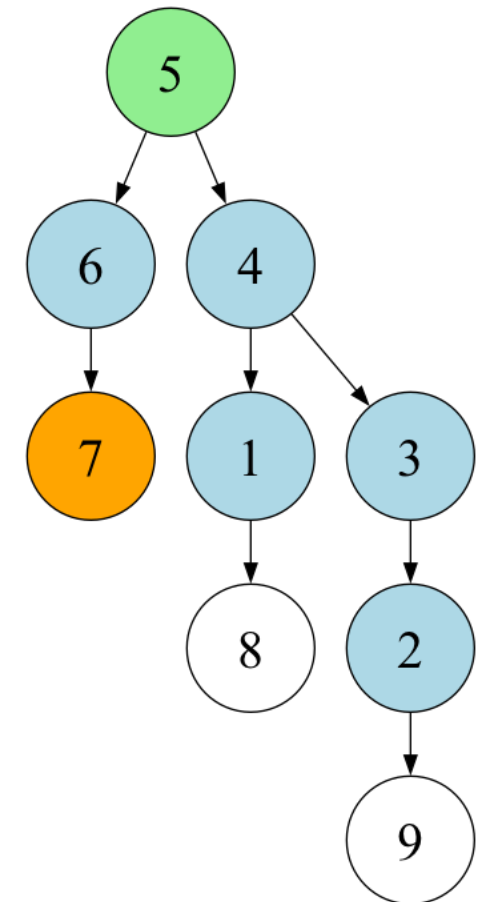
- Remove 6 from the stack S and check if it is the goal.
- It's not, so add it to the visited set V.
- Add unvisited neighbors of 6 to stack: 7
- Add 7 as child of 6 in tree.
- S: 6, 1, 3, 9, 8, **7**
- V: 5, 4, 3, 2, 1, **6**



Step 6: Exploring Node 6

How does this work on a graph?

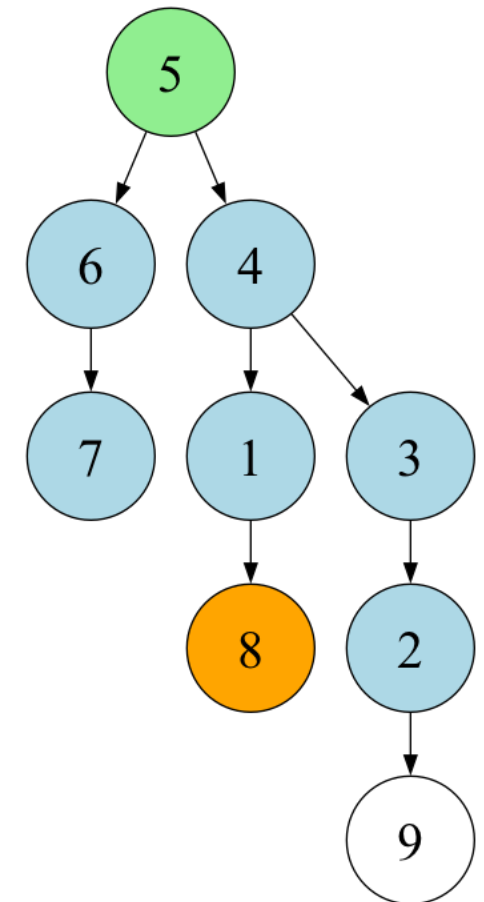
- Remove 7 from the stack S and check if it is the goal.
- It's not, so add it to the visited set V.
- Add unvisited neighbors of 7 to stack: 6, 8
- Add NOTHING as child of 7 in tree. (Why?)
- S: 6, 1, 3, 9, 8, **8**
- V: 5, 4, 3, 2, 1, 6, **7**



Step 7: Exploring Node 7

How does this work on a graph?

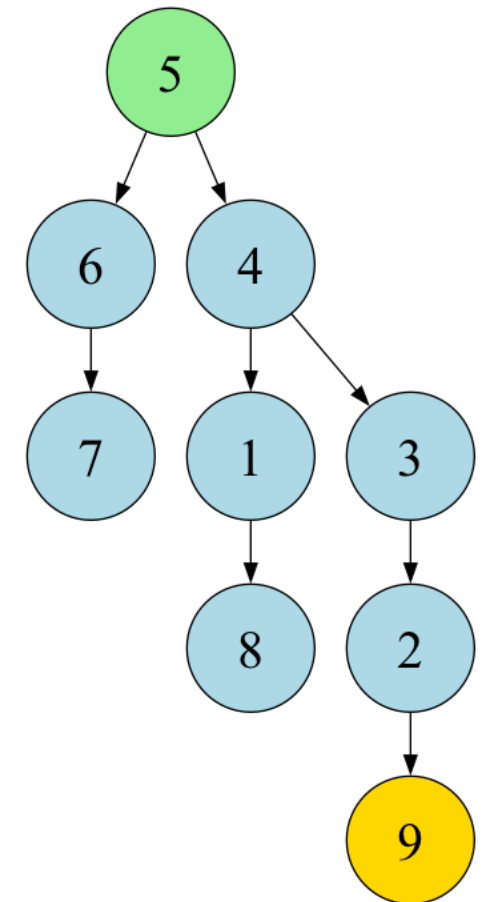
- Remove 8 from the stack S and check if it is the goal.
- It's not, so add it to the visited set V.
- Add unvisited neighbors of 8 to stack: 9
- Add NOTHING as child of 8 in tree. (Why?)
- S: 6, 1, 3, 9, 8, **9**
- V: 5, 4, 3, 2, 1, 6, 7, **8**



Step 8: Exploring Node 8

How does this work on a graph?

- Remove 9 from the stack S and check if it is the goal.
- **It is! So, let's end the loop!**



Step 9: Exploring Node 9

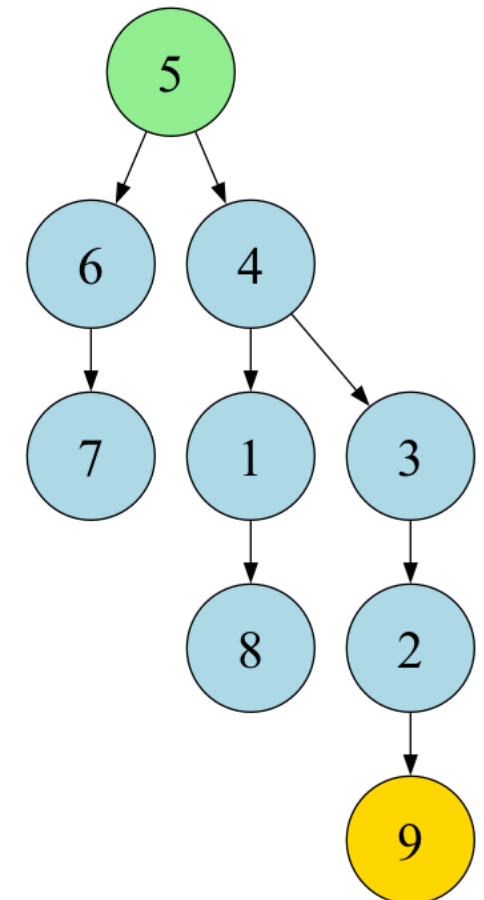
- S: 6, 1, 3, 9, 8
- V: 5, 4, 3, 2, 1, 6, 7, 8, **9**

How does this work on a graph?

Compare the path in the tree to the visited order...
they are NOT the same!

- Path (according to the tree): $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 9$
(We get the path via *backtracking* from 9 to 5).

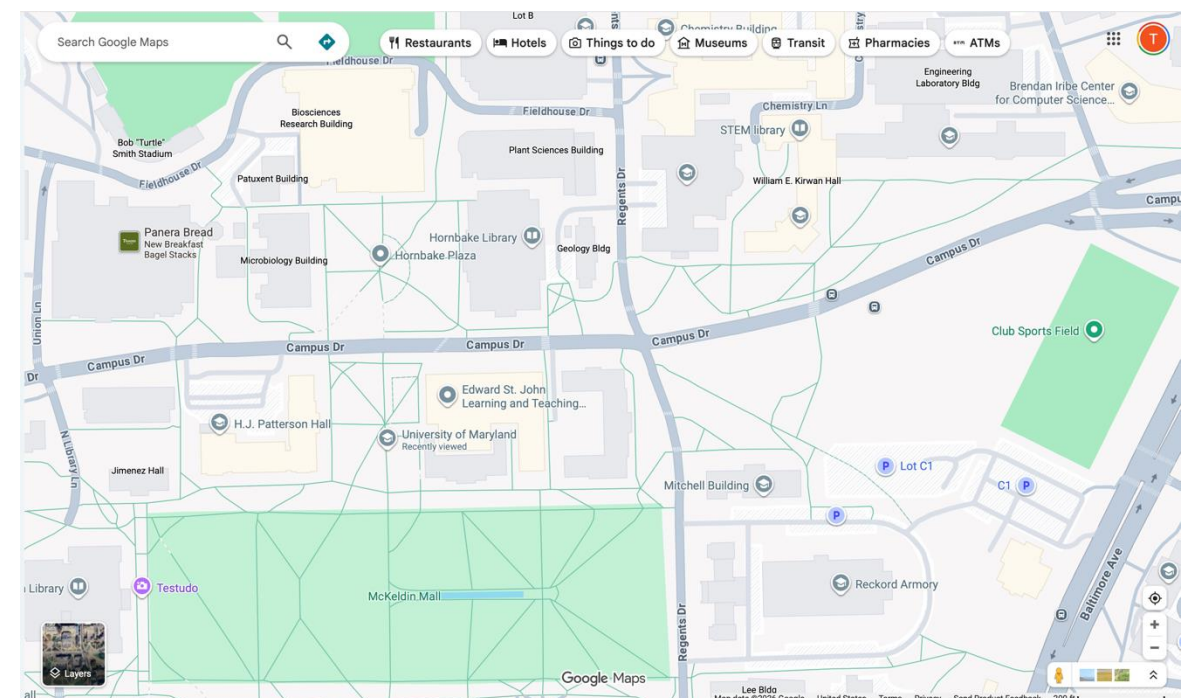
- S: 6, 1, 3, 9, 8
- V: 5, 4, 3, 2, 1, 6, 7, 8, **9**



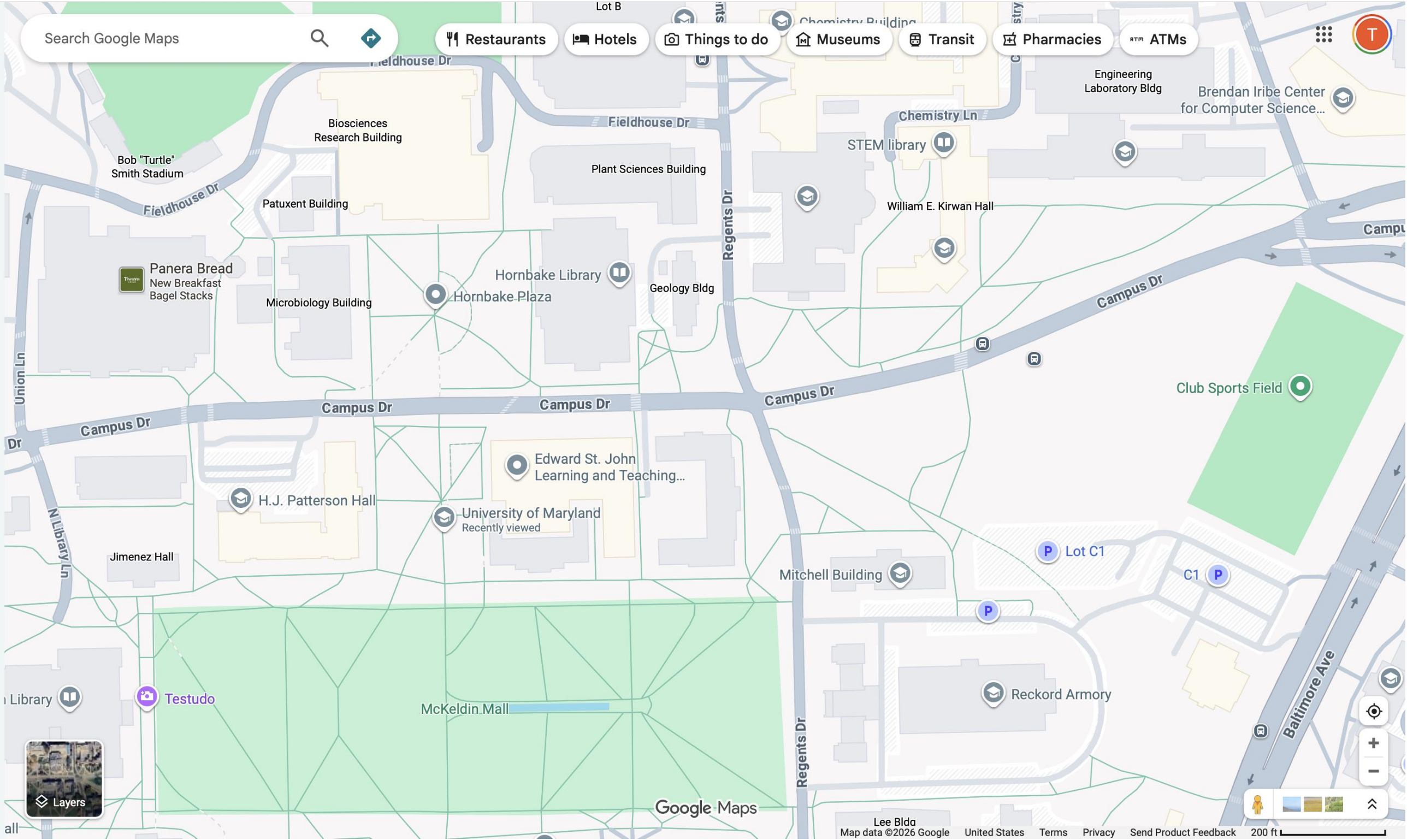
Step 9: Exploring Node 9

Activity—Path Planning

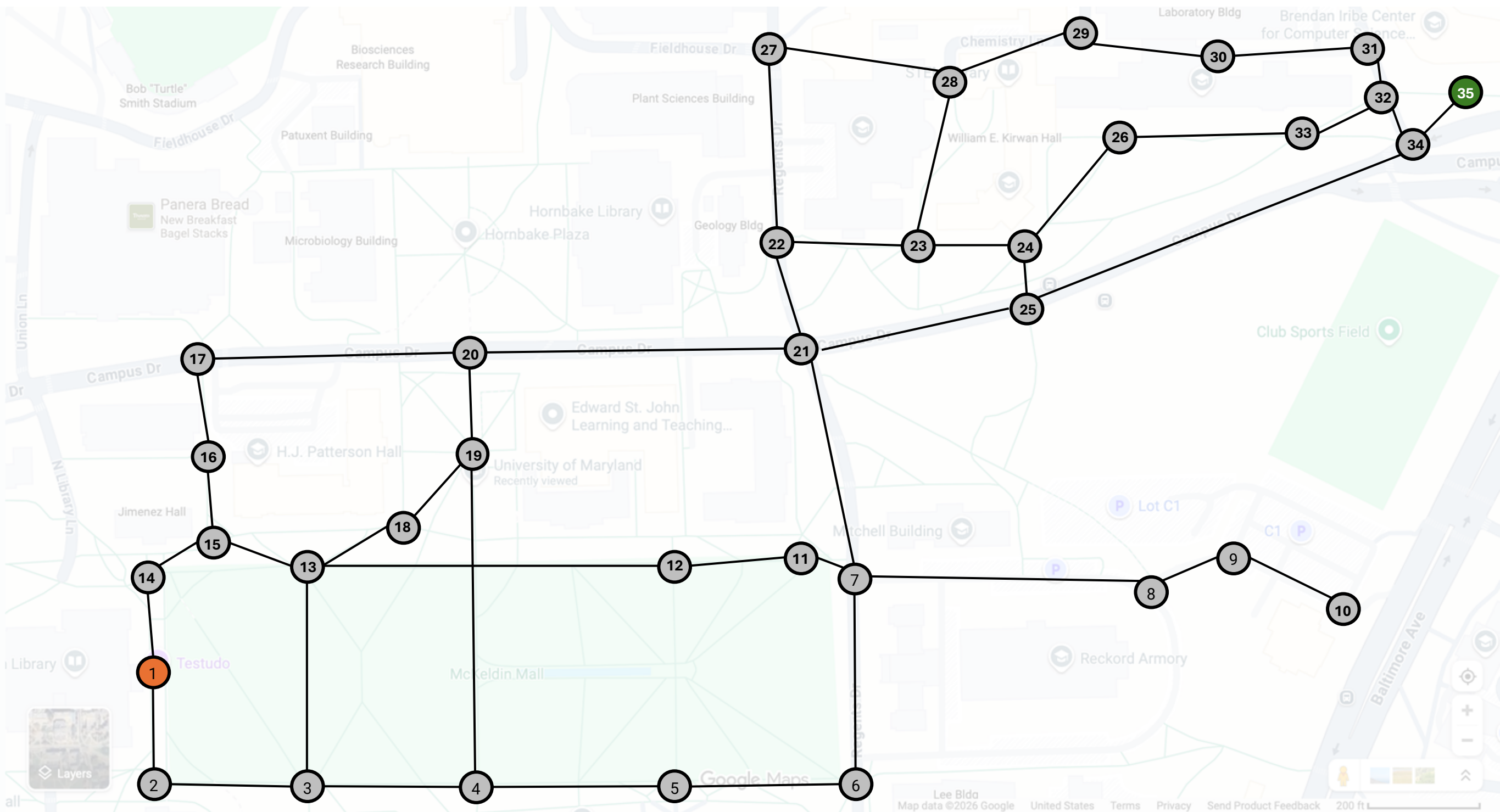
- **You are:** the path planning module for a food delivery robot.
- **Your Goal:** Plan a path from the Library to Iribe!
Let's start simple: plan the first 15 steps in class.
- **How?** Use **DFS** to find a path between the start and goal using a known map.
- **Order:** Clockwise, starting at the right (3 o'clock position).
- **Output:** The order in which you traveled from start to goal.



Activity—Path Planning

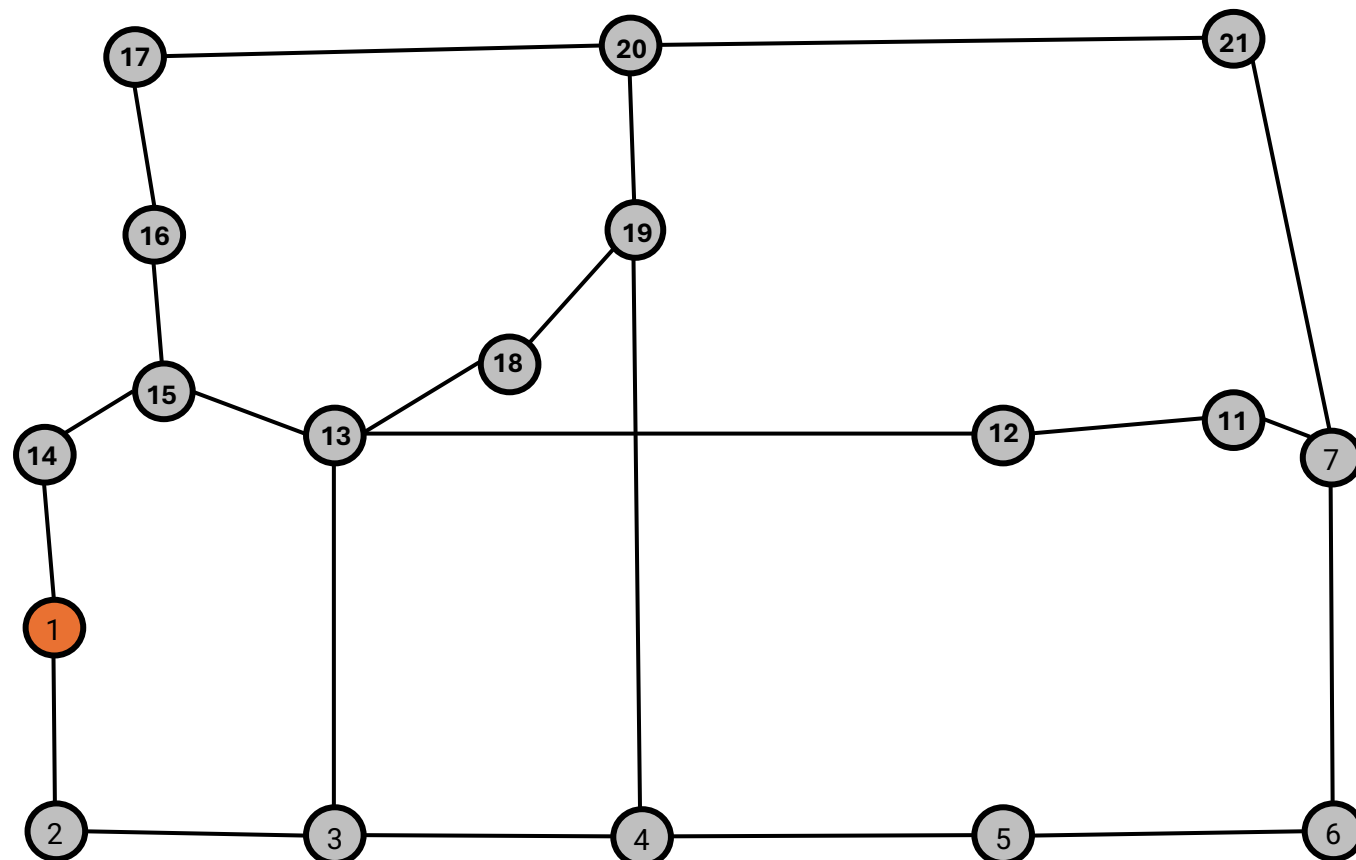


Activity—Path Planning

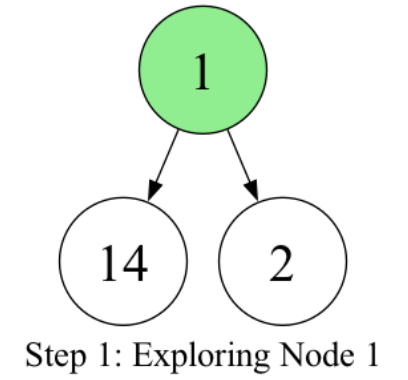


DFS tree after 0 steps

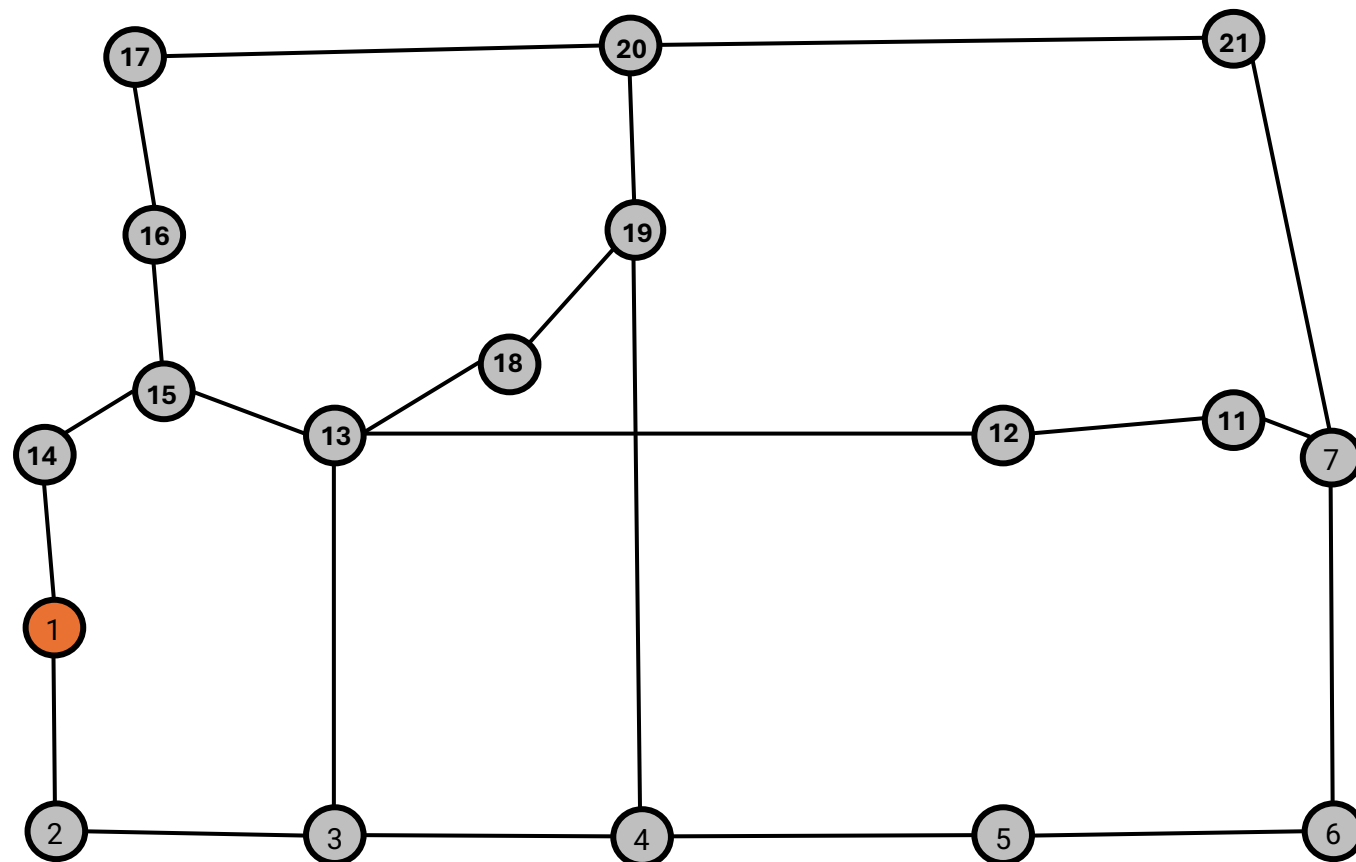
- S: 1
- V:



DFS tree after 1 steps

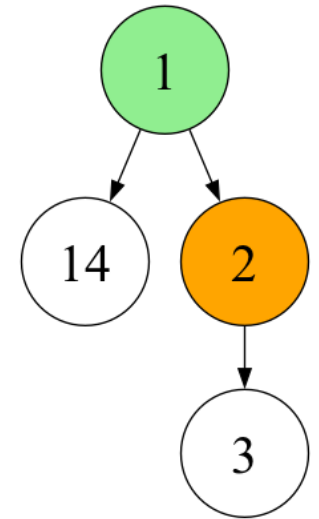


- S: 14, 2
- V: 1

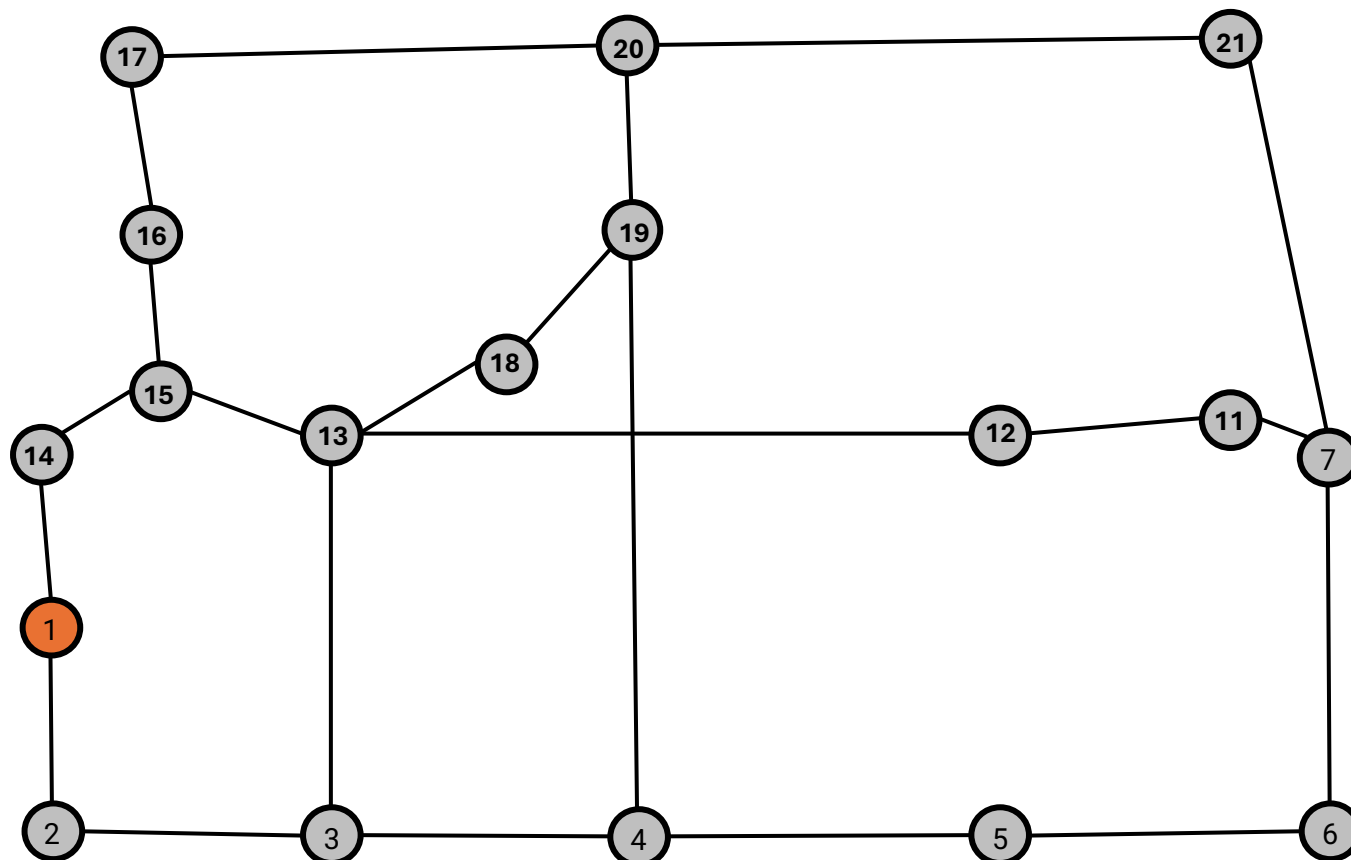


DFS tree after 2 steps

- S: 14, 3
- V: 1, 2

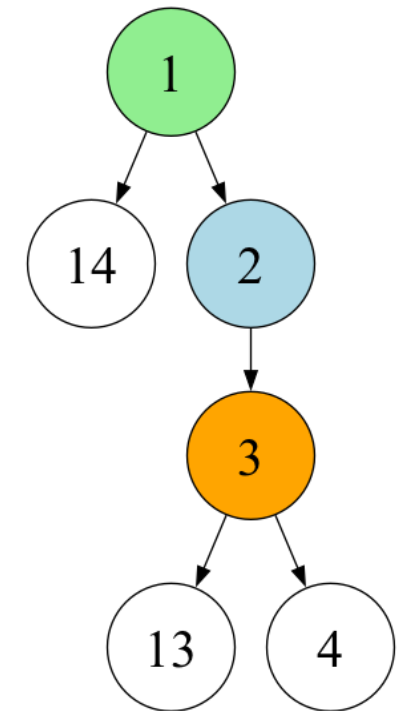


Step 2: Exploring Node 2

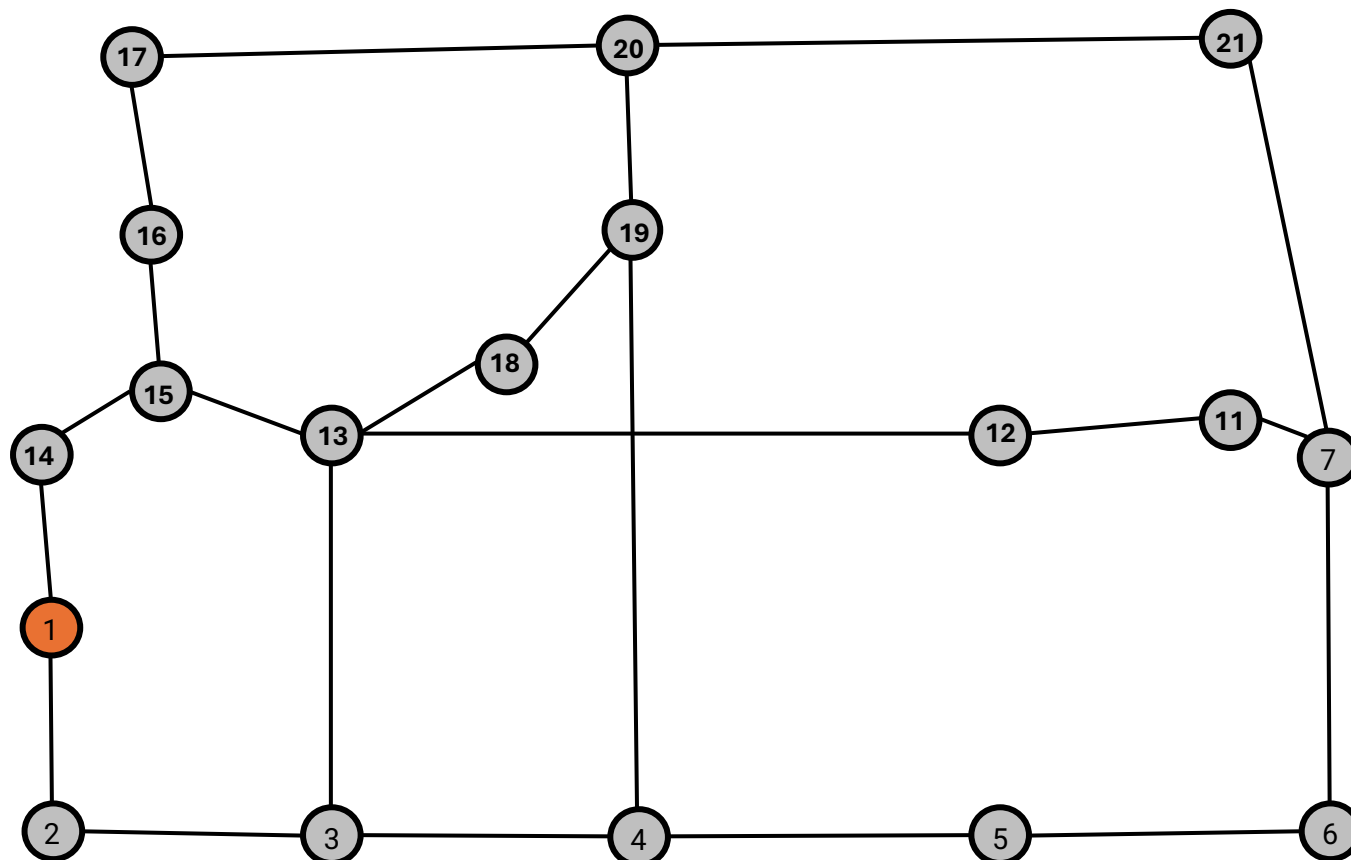


DFS tree after 3 steps

- S: 14, 13, 4
- V: 1, 2, 3

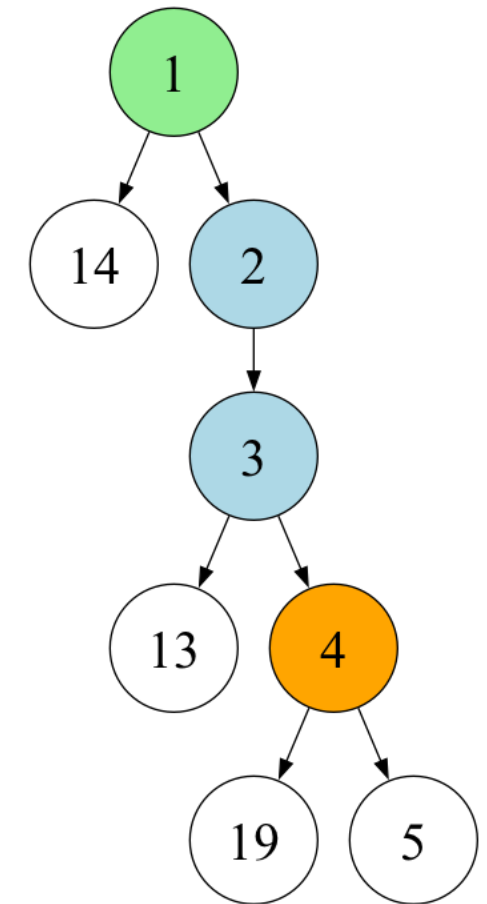


Step 3: Exploring Node 3

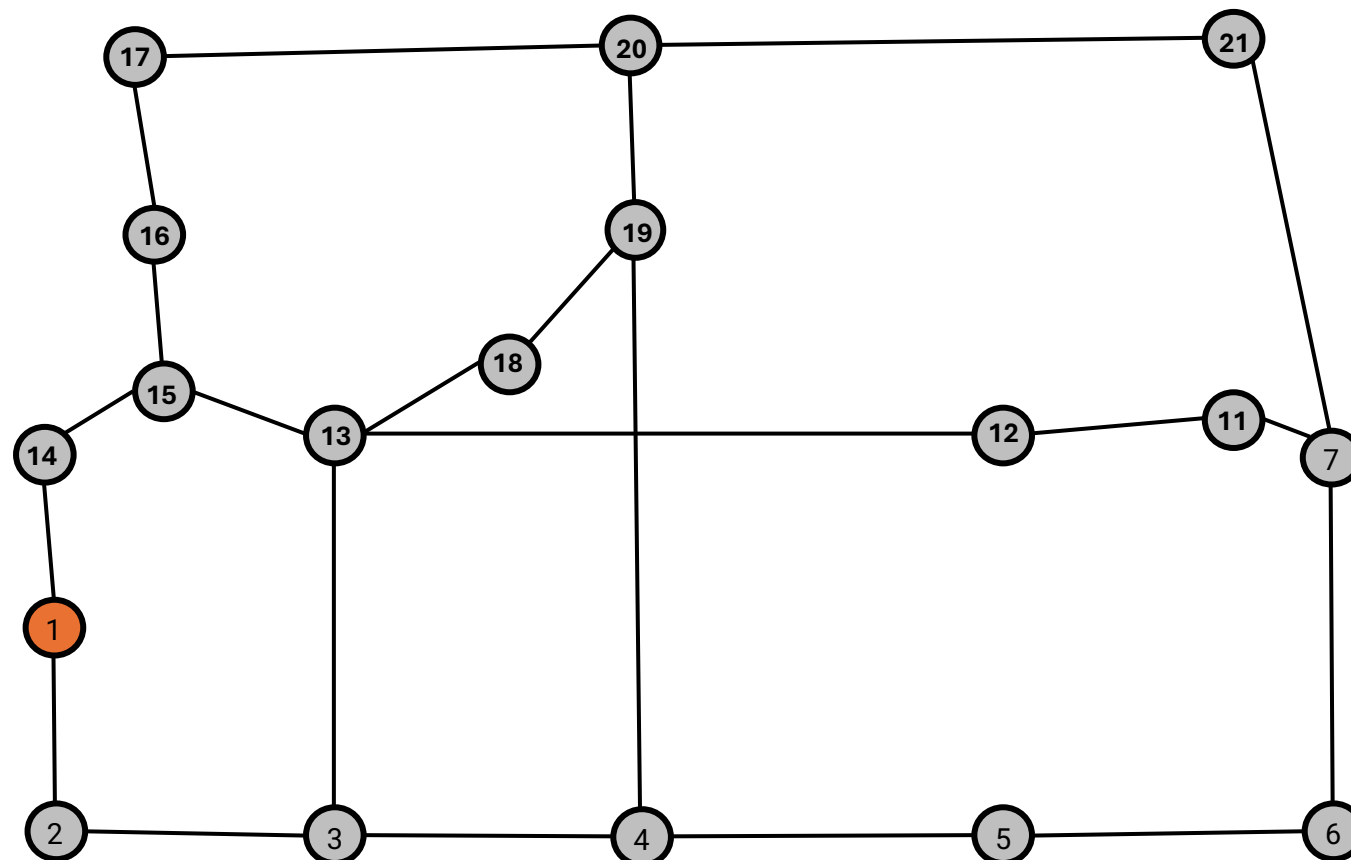


DFS tree after 4 steps

- S: 14, 13, 19, 5
- V: 1, 2, 3, 4

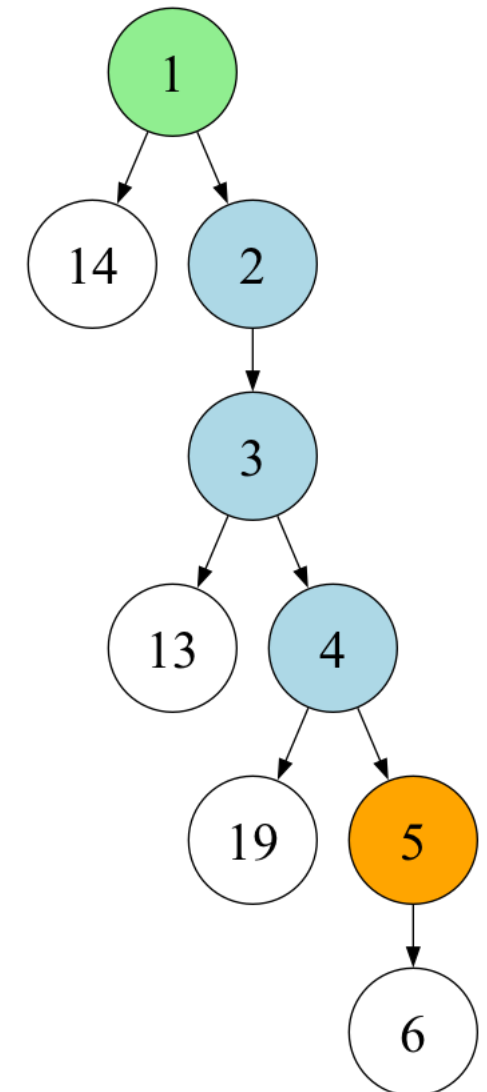
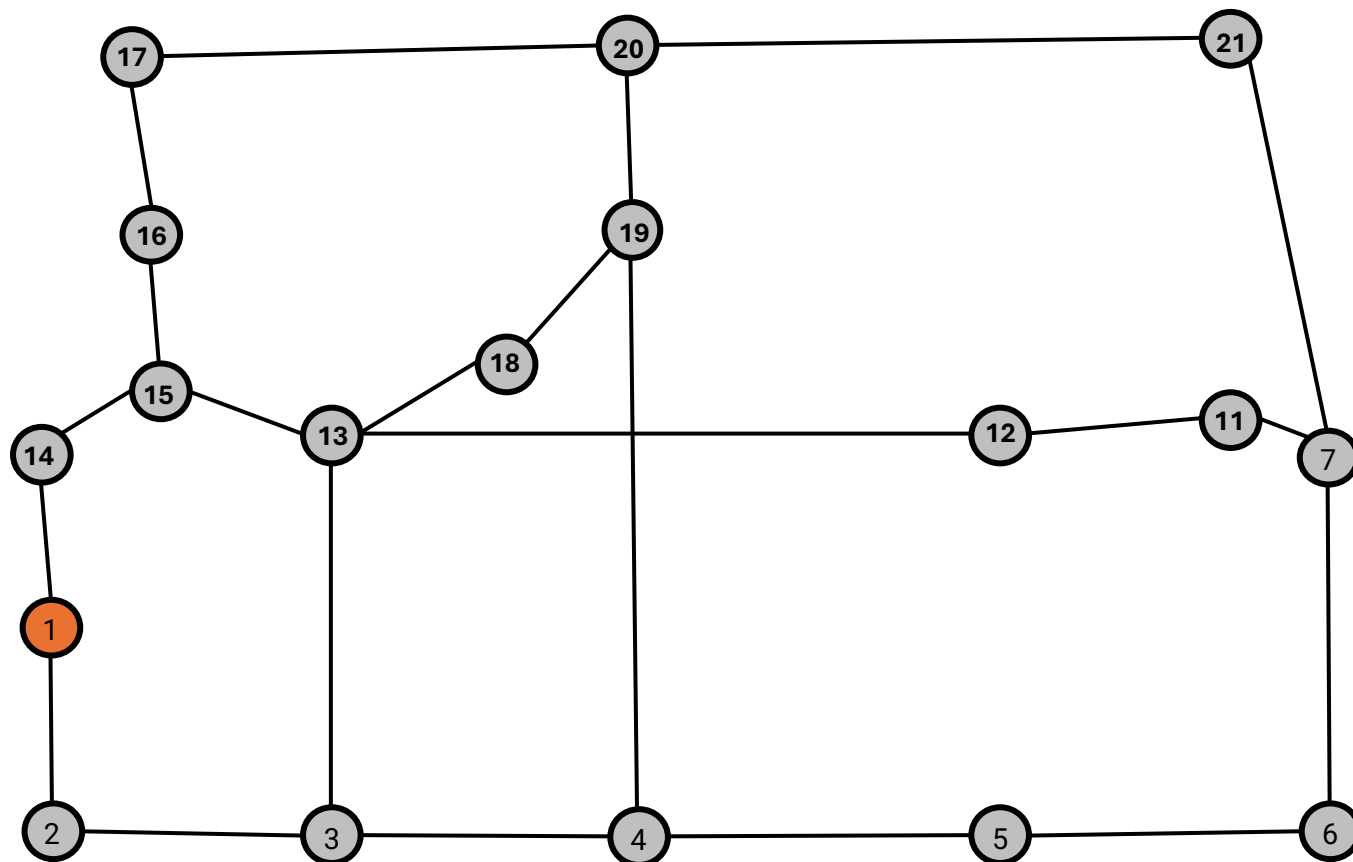


Step 4: Exploring Node 4



DFS tree after 5 steps

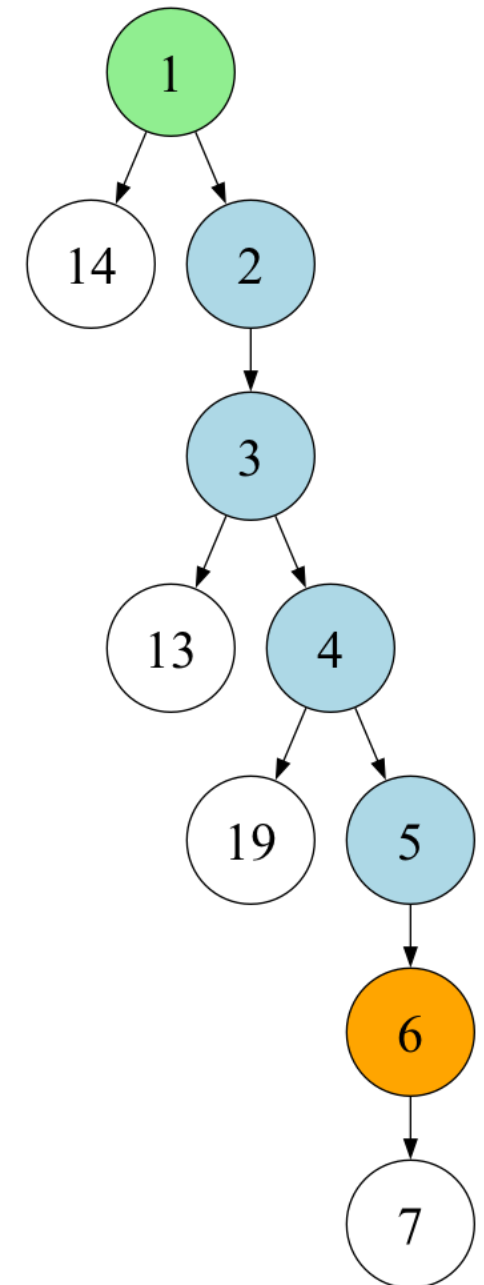
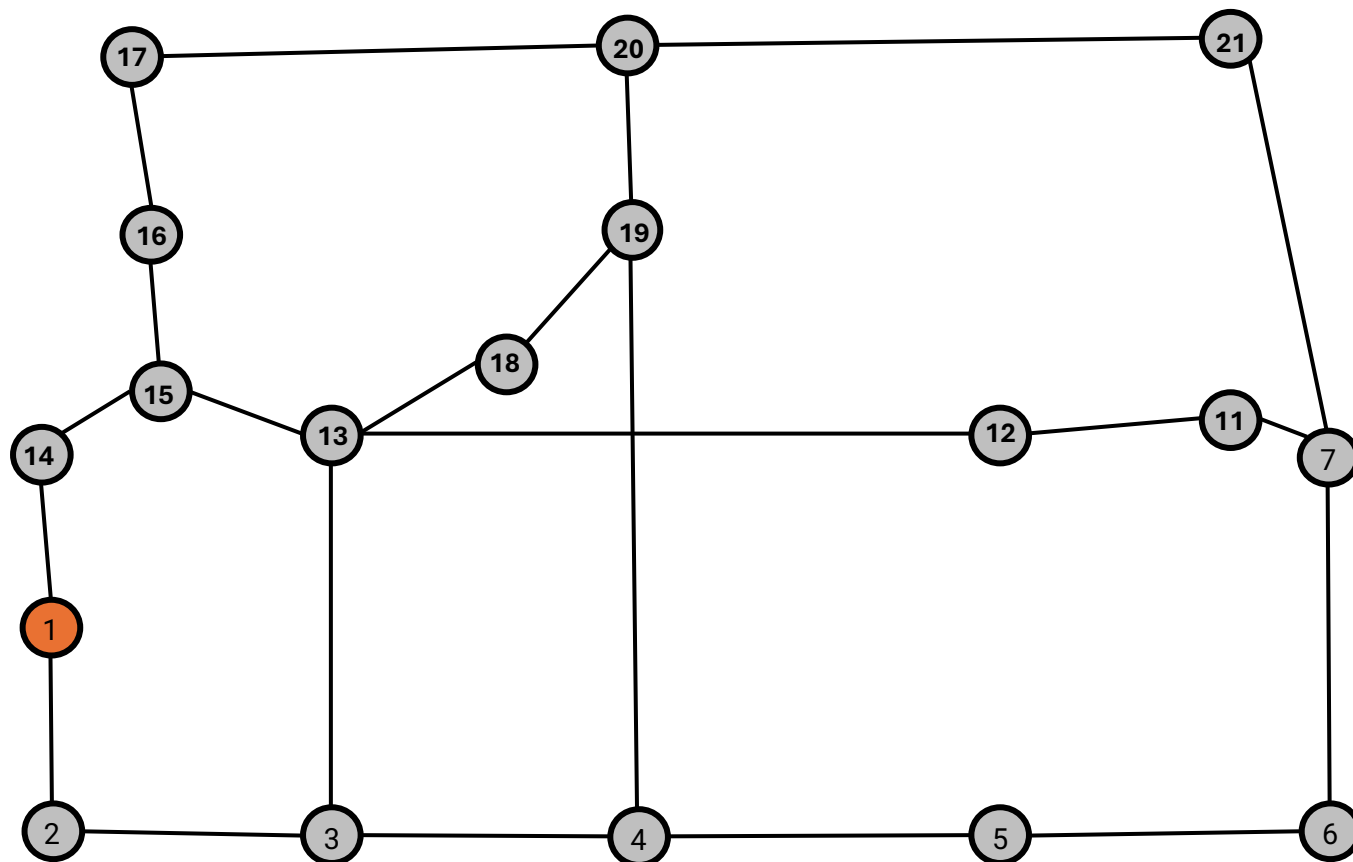
- S: 14, 13, 19, 6
- V: 1, 2, 3, 4, 5



Step 5: Exploring Node 5

DFS tree after 6 steps

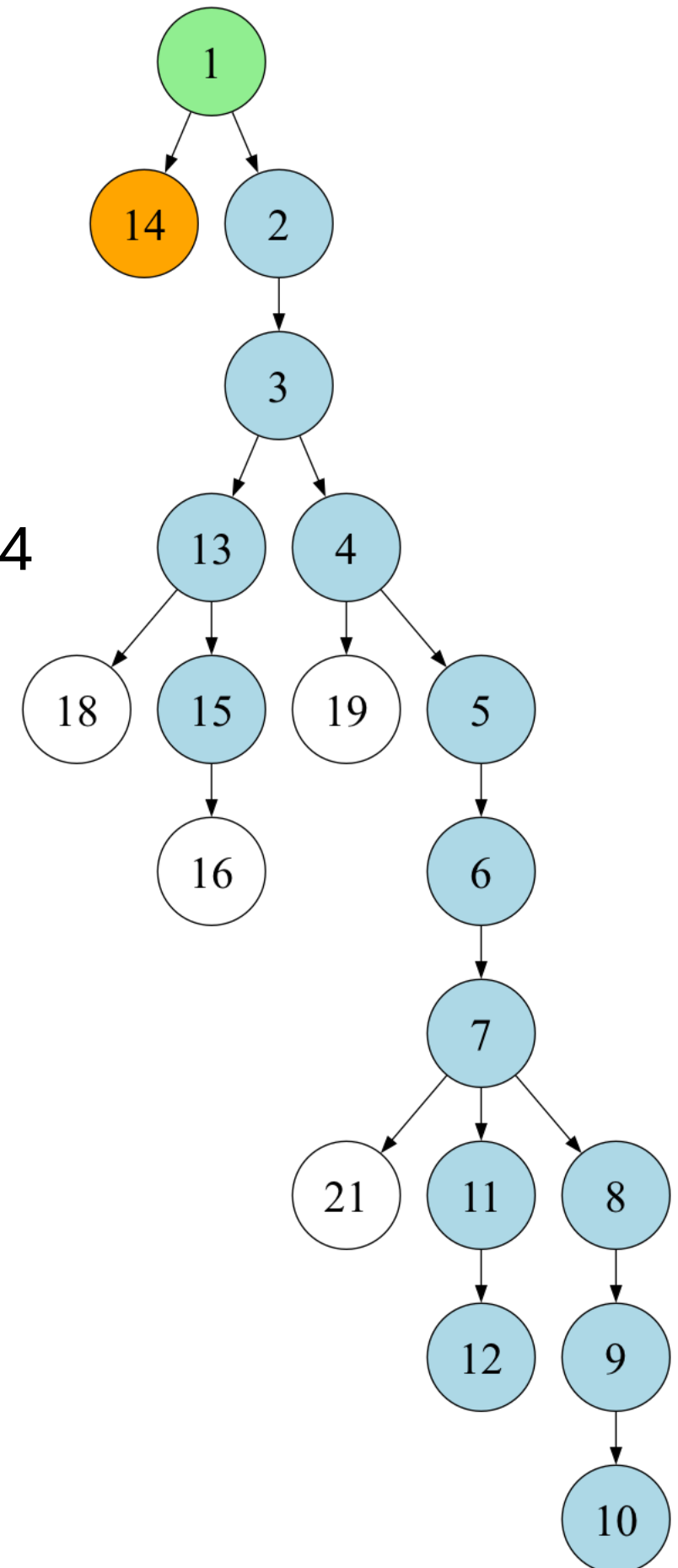
- S: 14, 13, 19, 7
- V: 1, 2, 3, 4, 5, 6



Step 6: Exploring Node 6

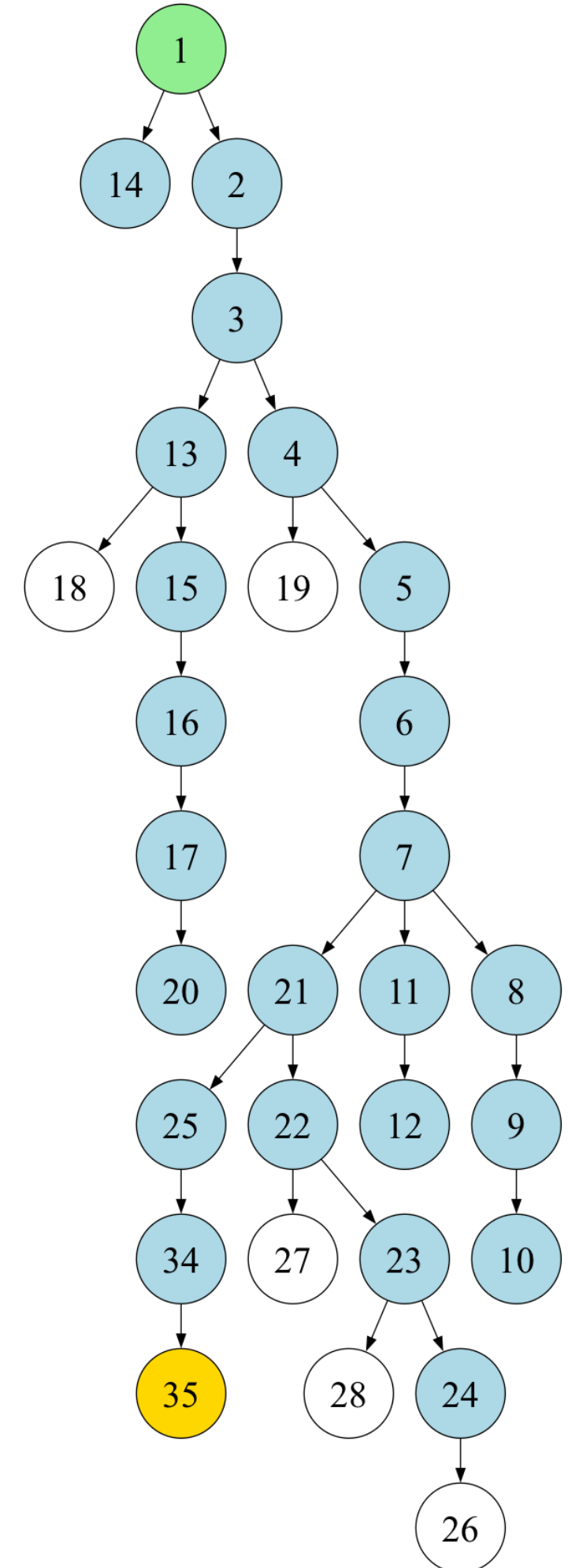
DFS tree after 15 steps

- S: 14, 13, 19, 21, 18, 16
- V: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 15, 14



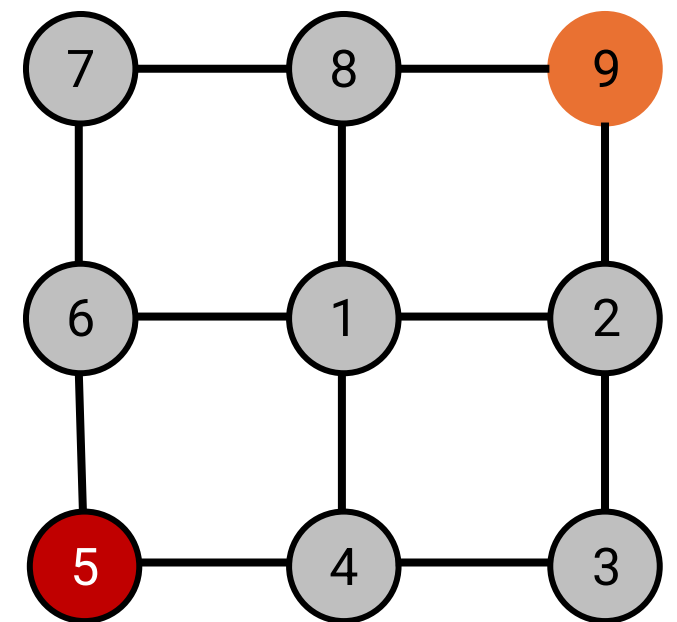
DFS tree at the end

- S: 14, 13, 19, 21, 18, 19, 25, 27, 28, 26
- V: 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 15, 14, 16, 17, 20, 21, 22, 23, 24, 25, 34, 35



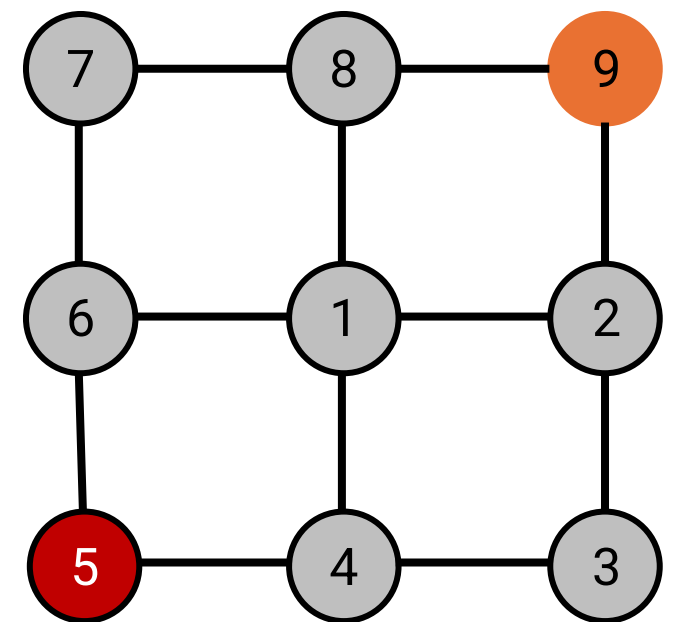
How does this work on a graph?

- Find a path from 5 to 9
- Use BFS
- Need to decide which order to add children nodes in
- Need to avoid cycles
- The subsequent slides will show the BFS output as a tree with (S) denoting the stack and (V) denoting the visited nodes.



How does this work on a graph?

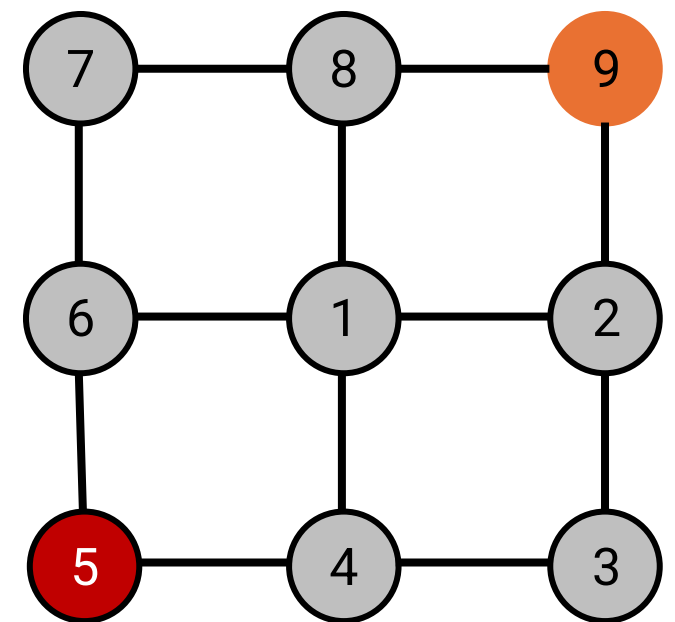
- Data structures used:
 - **Queue** to store the list of vertices to explore. A stack operates as a First-In, First-Out data structure. (Here's a [YouTube video on queues](#)).
 - **Set** to maintain the vertices we've visited (i.e., the vertices we've removed from the queue to explore).
 - **Tree** to store the final path from the start vertex to the goal vertex. *At the end, you'll see why we use a tree to find the path.
- Exploration order: Right, down, left, up.



How does this work on a graph?

Algorithm steps:

1. Add the start vertex to the queue.
2. Loop while your queue Q is not empty:
 - a. Remove vertex X from the **top of the queue Q** (the last vertex you added).
 - b. If you've visited X before (i.e., X is in your visited set V), go to step 2a.
 - c. **Add X to your visited set V .**
 - d. If X is your goal, exit the loop!
 - e. Find all the **unvisited neighbors of X** and **add them to the queue** (remember the explore order).
 - f. **Add neighbor C under X** in the tree if C is not on the queue Q or in the visited set V .

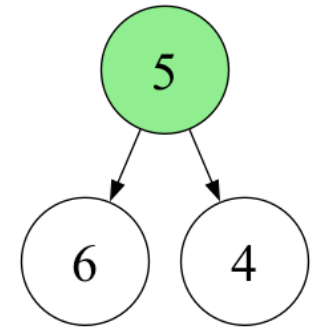


How does this work on a graph?

- Start with node 5 on the queue
(initialization).

- Q: 5
- V:

How does this work on a graph?

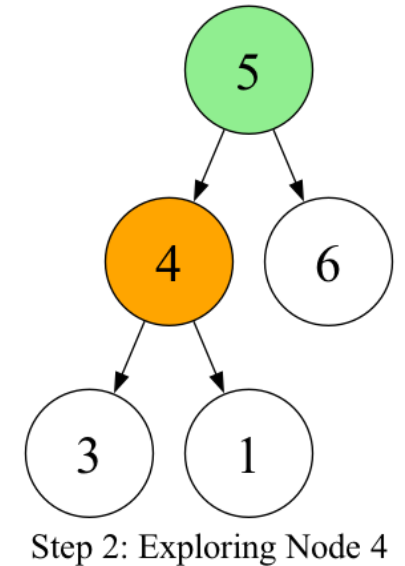


Step 1: Exploring Node 5

- Remove 5 from the queue Q and check if it is the goal.
- It's not, so add it to the visited set V.
- Add unvisited neighbors of 5 to queue: 4, 6
- Add 4, 6 as children of 5 in tree.
- Q: 4, 6
- V: 5

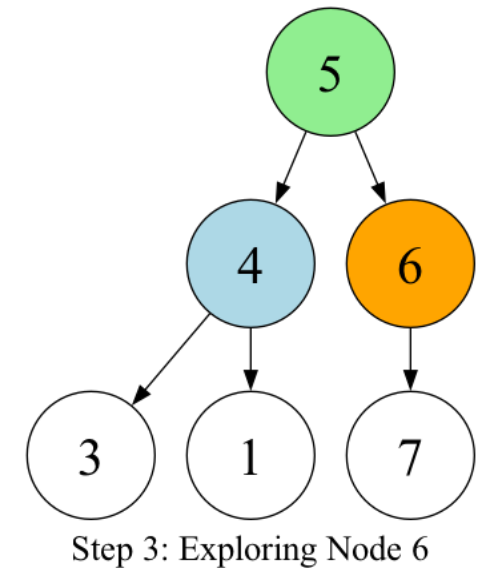
How does this work on a graph?

- Remove 4 from the queue Q and check if it is the goal.
- It's not, so add it to the visited set V.
- Add unvisited neighbors of 4 to queue: 3, 1
- Add 3, 1 as children of 4 in tree.
- Q: 6, 3, 1
- V: 5, 4



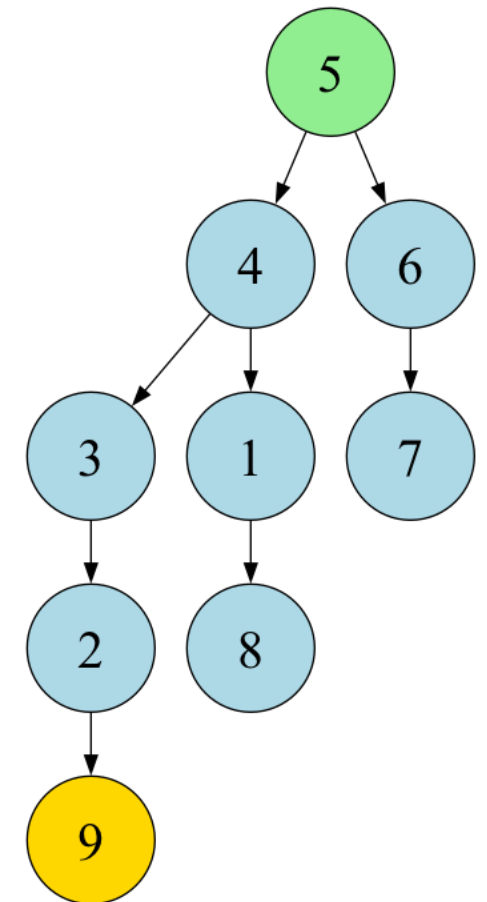
How does this work on a graph?

- Remove 6 from the queue Q and check if it is the goal.
- It's not, so add it to the visited set V.
- Add unvisited neighbors of 6 to queue: 7
- Add 7 as child of 6 in tree.
- Q: 3, 1, 7
- V: 5, 4, 6



How does this work on a graph?

- Can you finish the rest to get?
- Q:
- V: 5, 4, 6, 3, 1, 7, 2, 8, 9



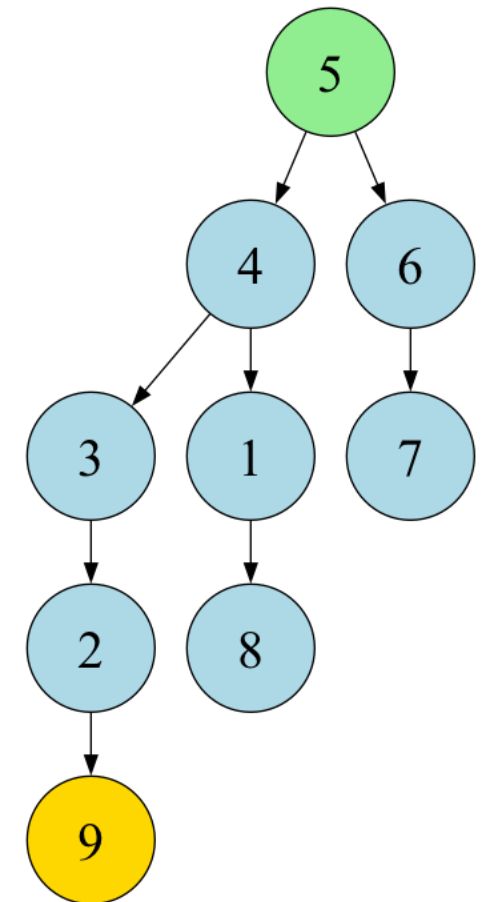
Step 14: Exploring Node 9

How does this work on a graph?

Compare the path in the tree to the visited order...
they are NOT the same!

- Path (according to the tree): $5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 9$
(We get the path via *backtracking* from 9 to 5).

- Q:
- V: 5, 4, 6, 3, 1, 7, 2, 8, 9



Step 14: Exploring Node 9